Alberto Andrés Valdés González.
Degree: Mathematical Engineer.
Work position: ML-Engineer.
Mail: anvaldes@uc.cl/alberto.valdes.gonzalez.96@gmail.com
Location: Santiago, Chile.

CI / CD / CT: differences and tools

Setting: A traditional software system changes only when its **code** changes. An ML system changes when any of three things change: the **code**, the **model** or the **data**. Each practice automates the reaction to one of those changes:

$$\underbrace{\text{CI}}_{\text{code changed}} \longrightarrow \underbrace{\text{CD}}_{\text{artifact ready}} \longrightarrow \underbrace{\text{CT}}_{\text{data / world changed}} \circlearrowleft$$

- **CI (Continuous Integration):** every change pushed to the repository is automatically built and tested. *Question it answers:* "does the new code work and not break anything?"
- **CD (Continuous Delivery / Deployment):** every artifact that passes CI is automatically packaged and released to an environment. *Question:* "can this version reach users safely?"
- **CT (Continuous Training):** the model is automatically retrained, evaluated and registered when new data arrives or the data distribution drifts. *Question:* "is the model in production still good for today's data?"

The full loop:

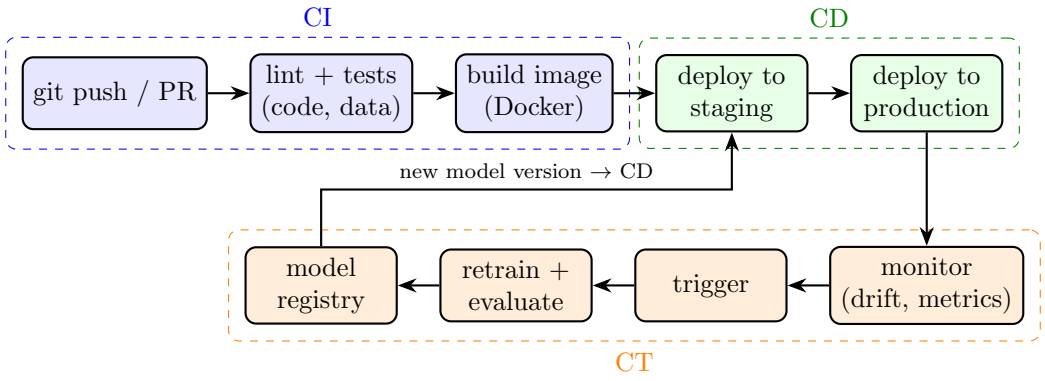


Figure 1: CI validates code, CD ships artifacts, CT keeps the model fresh and feeds new versions back into CD.

1. CI: Continuous Integration

Trigger: a commit or pull request.
Goal: merge small changes often, detecting errors in minutes instead of weeks.
Typical steps:

(a) Static checks: formatting, linting, type checking.

(b) Unit and integration tests of the code.

(c) **ML-specific:** data validation (schema, nulls, ranges), tests of feature transformations, and a smoke training on a small sample to check the pipeline runs end to end.

(d) Build the artifact (Docker image, Python wheel) and push it to a registry.

Output: a tested, versioned artifact.

2. CD: Continuous Delivery vs Continuous Deployment

Both automate the release; they differ only in the last step:

$$\begin{aligned} \text{Delivery:} &\quad \text{CI} \to \text{staging} \to \boxed{\text{manual approval}} \to \text{production} \\ \text{Deployment:} &\quad \text{CI} \to \text{staging} \to \text{production} \quad \text{(no human in the loop)} \end{aligned}$$

Trigger: an artifact that passed CI (or a new model version promoted in the registry).
Typical steps:

(a) Deploy to staging and run integration / load tests.

(b) Release strategy: **blue-green** (switch all traffic at once, instant rollback), **canary** (send 5% of traffic, then increase), or **shadow** (new model receives real traffic but its answers are not returned, only logged and compared).

(c) Automatic rollback if latency, error rate or business metrics degrade.

Output: a running service (an API endpoint, a batch job) in production.

3. CT: Continuous Training

CT exists only in ML. A model can be perfectly coded and deployed and still degrade, because the world changes (**data drift**: $P(X)$ changes; **concept drift**: $P(Y \mid X)$ changes).

Triggers:

- **Schedule:** retrain every day / week.
- **New data volume:** retrain after N new labeled samples.

- **Performance drop:** a monitored metric (e.g. AUC on recent labeled data) falls below a threshold.
- **Drift:** the distribution of a feature moves. A common score is the Population Stability Index, comparing k bins of the training distribution p_j with production q_j:

$$PSI = \sum_{j=1}^{k} (q_j - p_j) \cdot \ln\left(\frac{q_j}{p_j}\right)$$

$PSI < 0.1$: stable, $0.1 \leq PSI < 0.25$: moderate, $PSI \geq 0.25$: significant drift, retrain

Typical steps: extract fresh data → validate it → train → evaluate against the current production model on the same holdout → register the new version only if it is better (champion / challenger) → hand it to CD.

Output: a new, evaluated model version in the registry.

Comparison:

	CI	**CD**	**CT**
Reacts to	Code changes	A validated artifact / model version	New data, drift, metric drop, schedule
Tests	Code, data schema, pipeline smoke run	Integration, load, latency, rollout health	Data quality, model metrics vs champion
Output	Versioned artifact (image, package)	Service running in production	New model version in registry
Frequency	Every commit / PR	Every approved artifact	Hours to weeks
Exists in classic software?	Yes	Yes	No, ML only

Tools by practice:

	CI	**CD**	**CT**
Orchestration / runners	GitHub Actions, GitLab CI, Jenkins, CircleCI, Azure Pipelines	Argo CD, Flux (GitOps), Spinnaker, AWS CodePipeline, Cloud Deploy	Airflow, Kubeflow Pipelines, Vertex AI Pipelines, SageMaker Pipelines, Prefect, Dagster
Quality / tests	pytest, ruff, mypy, pre-commit, Great Expectations, Pandera	Locust, k6 (load tests), Argo Rollouts / Flagger (canary analysis)	Great Expectations, TFX Data Validation, Deepchecks
Packaging / infra	Docker, Poetry / uv, container registries (ECR, GCR, Artifact Registry)	Kubernetes, Helm, Terraform, KServe, Seldon Core, BentoML, SageMaker / Vertex endpoints	Feature stores (Feast, Vertex / SageMaker Feature Store)
Versioning	Git, DVC (data and model versioning)	Git (GitOps: the repo is the desired state)	MLflow (tracking + model registry), Weights & Biases, DVC
Monitoring / triggers	–	Prometheus, Grafana, Datadog	Evidently, WhyLabs, Arize, NannyML (drift and performance)

Note: many tools cover more than one practice. GitHub Actions can run CI and trigger deployments; Vertex AI and SageMaker cover CT and CD end to end. The table shows where each tool is *most* used.

Maturity levels (Google MLOps):

- **Level 0, manual:** notebooks, manual training and manual deployment. No CI/CD/CT.
- **Level 1, pipeline automation:** the training pipeline is automated, so **CT** exists; the pipeline code itself is still deployed manually.
- **Level 2, CI/CD pipeline automation:** the pipeline code goes through **CI** and **CD**, and the deployed pipeline performs **CT**. All three loops are automated.

Summary:

CI answers "does the code work?" on every commit. **CD** answers "can this version reach users safely?" and ships it (with manual approval in Delivery, fully automatic in Deployment). **CT** answers "is the model still good for today's data?" and retrains it when data, drift or metrics say so, feeding the new version back into CD. CI and CD come from classic software; CT is what makes MLOps different.
